INFORME DE CALIDAD DE PRODUCTO

SPRINT 1

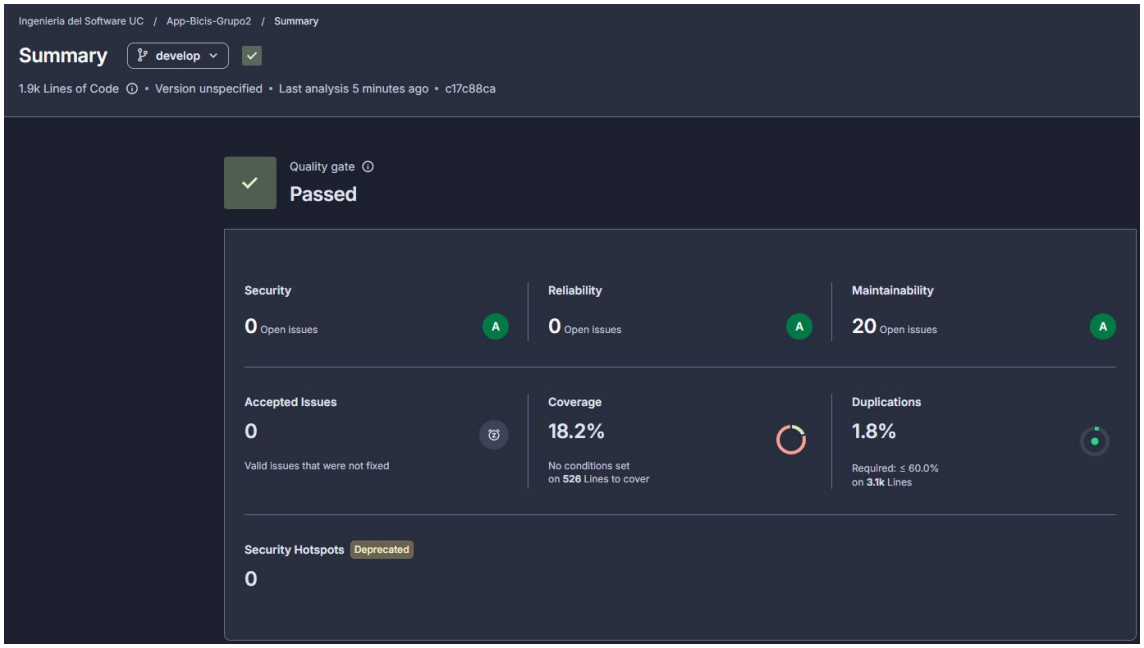
- **Proyecto:** App-Bicis
- **Grupo:** 2
- **Repositorio:** isunican/App-Bicis-Grupo2
- **Rama analizada:** develop
- **Commit analizado:** c17c88ca
- **Fecha de análisis:** 22/10/2026

Tabla de contenidos

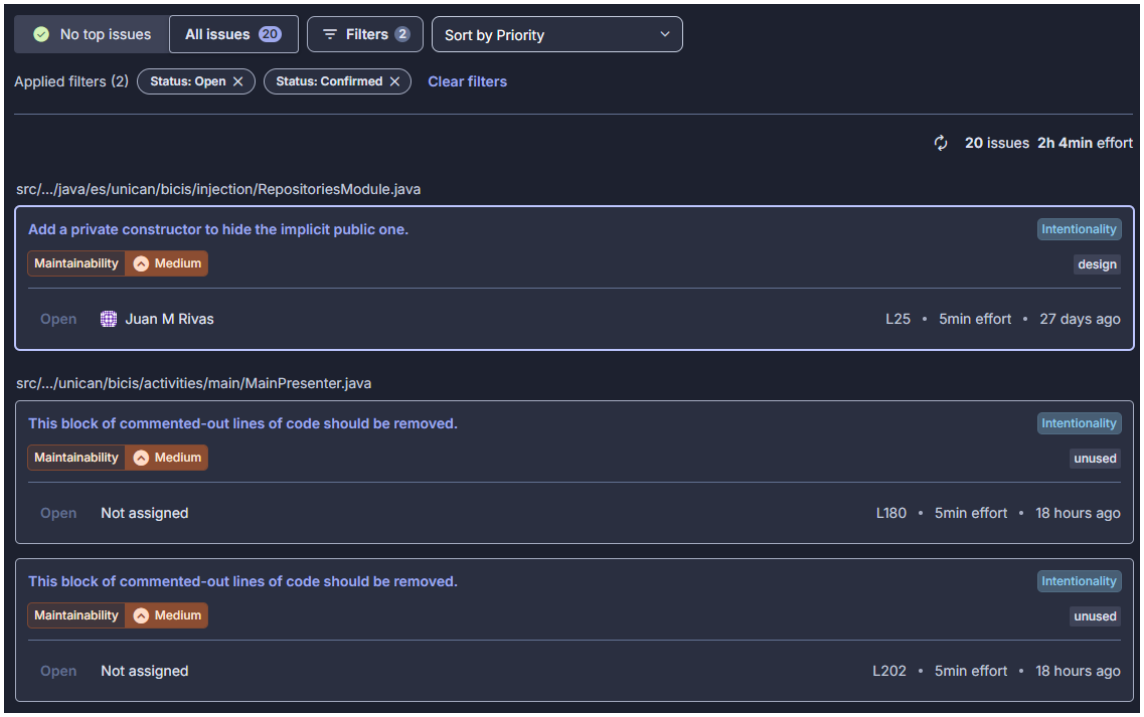
Datos obtenidos SonarCloud y ScrumDesk 3
Análisis del estado del producto 7
Plan de acción priorizado 8
Tareas añadidas al Sprint 10

Datos obtenidos SonarCloud y ScrumDesk

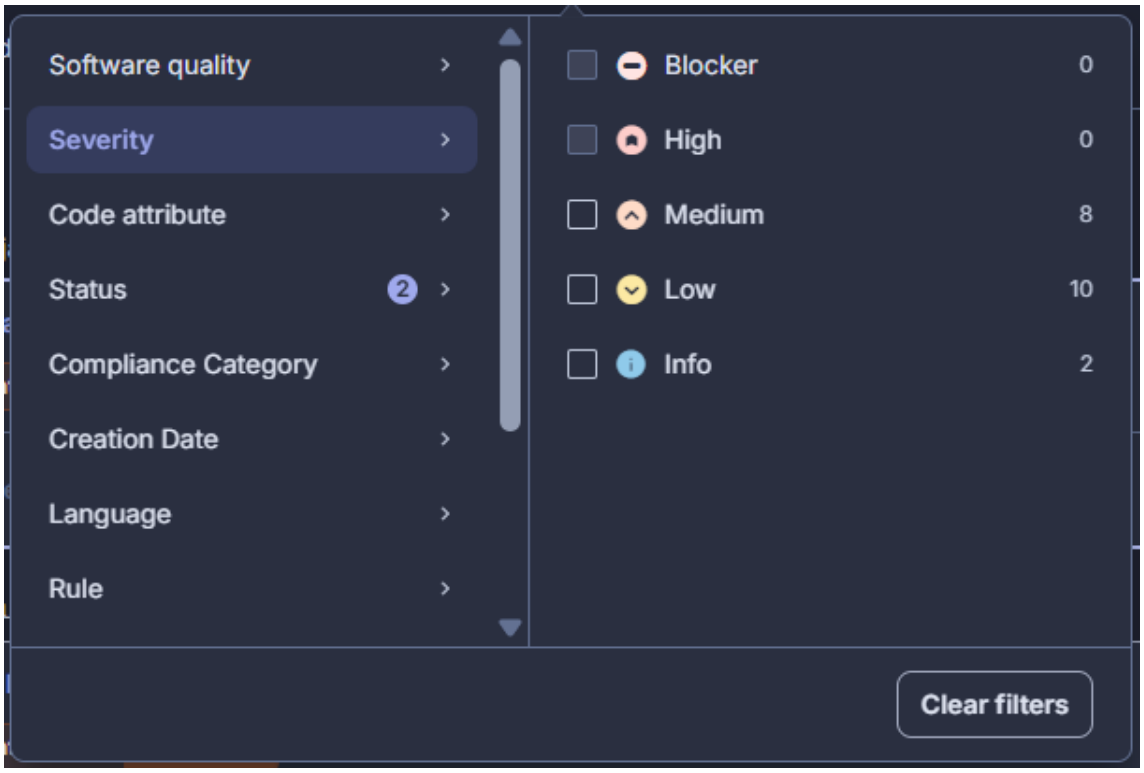
A continuación, se muestran capturas de los datos obtenidos tras analizar la rama develop con las historias de usuario implementadas en el sprint 1:



Captura 1: Resumen del análisis en SonarCloud



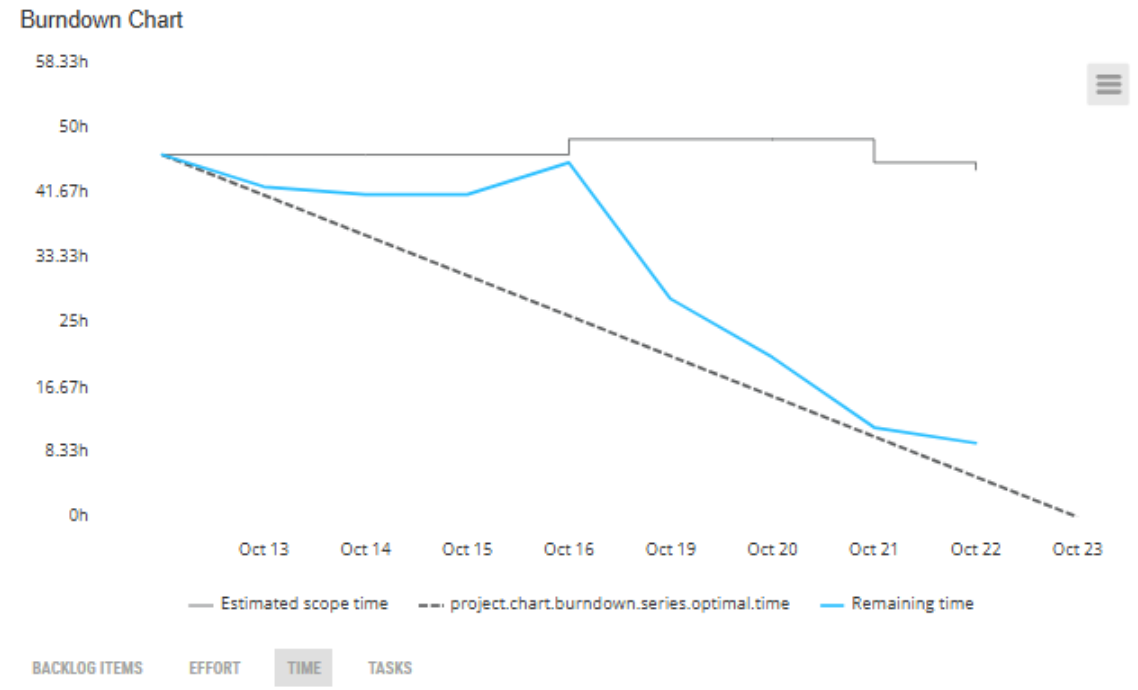
Captura 2: Listado de incidencias



Captura 3: Incidencias filtradas por severidad



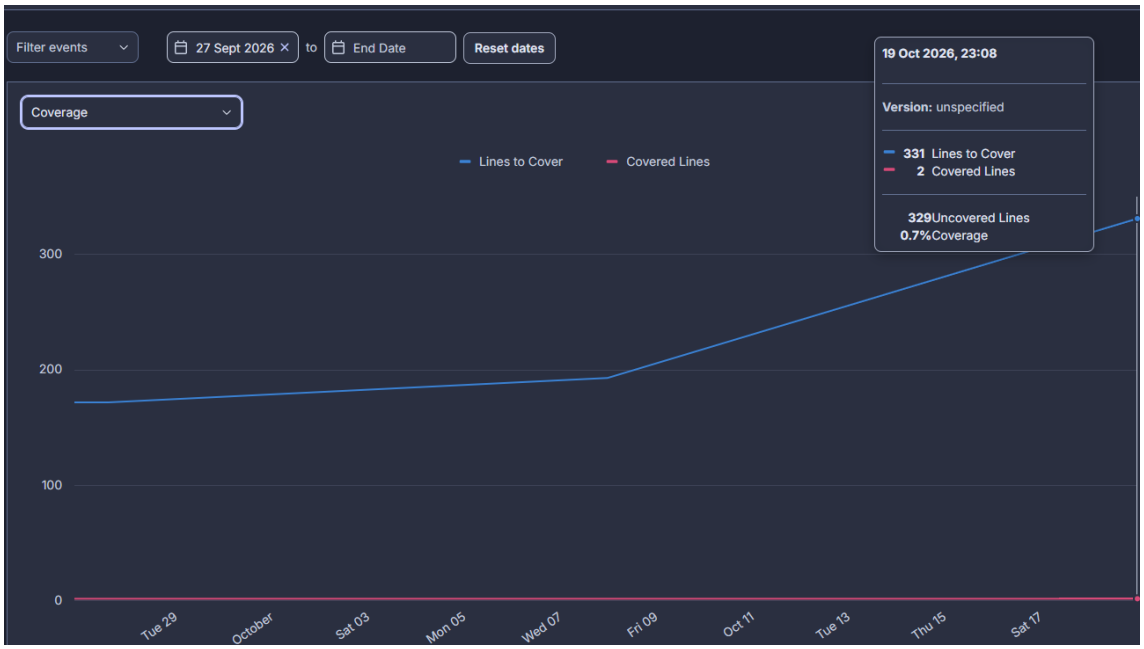
Captura 4: Métricas clave



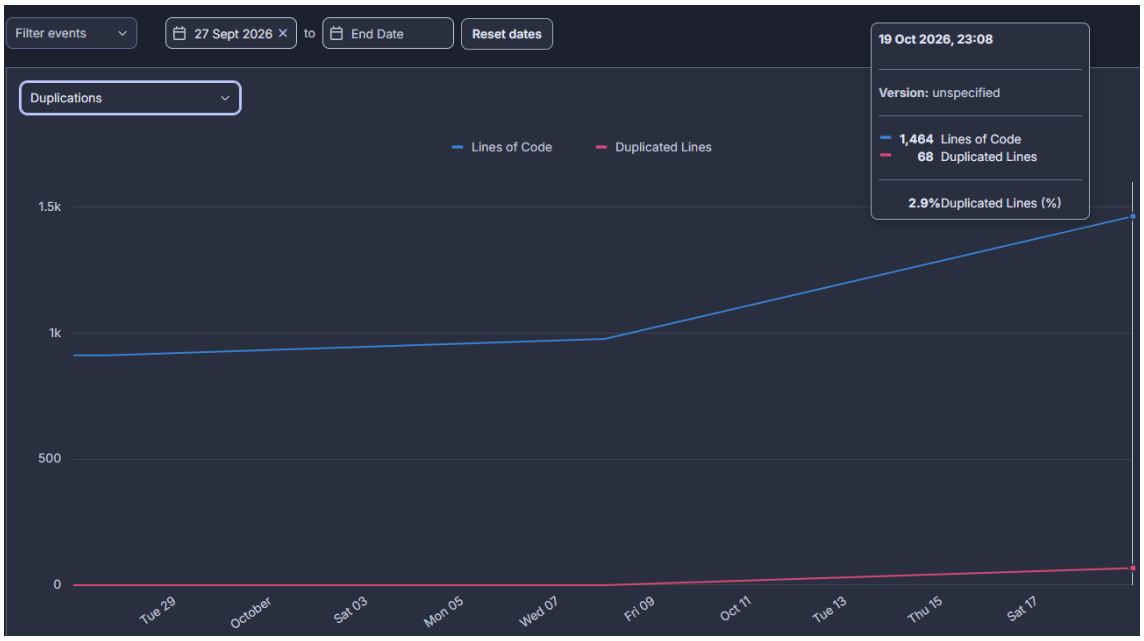
Captura 5: BurnDown Chart – Informe del sprint 1



Captura 6: Grafica evolución temporal 'Issues'



Captura 7: Grafica evolución temporal ‘Coverage’



Captura 8: Grafica evolución temporal ‘Duplications

Análisis del estado del producto

El análisis del código realizado sobre la rama 'develop'(commit c17c88ca) devuelve un estado aceptable en general, y que ha superado la inspección de calidad (como se muestra en la Captura 1), pero hay varias áreas de mejora en cuanto a la mantenibilidad y cobertura de pruebas unitarias.

Sobre los puntos fuertes del producto:

- Tiene un porcentaje muy bajo de duplicidad de código (1.8%) respecto al límite máximo permitido (60%) [Capturas 1 y 8].
- Todas las pruebas unitarias existentes se ejecutan correctamente sin errores ni fallos [Captura 4]
- El proyecto no muestra vulnerabilidades, ni hotspots de seguridad ni bugs, demostrando que es un producto seguro y fiable [Capturas 1 y 3].

A pesar de los buenos resultados que muestra SonarCloud, el producto tiene algunas áreas de mejora y algunos riesgos que ha detectado:

- **Baja cobertura de pruebas [Capturas 1 , 4 y 7]:** la cobertura de pruebas es insuficiente (18.2%). De las 526 líneas evaluables para pruebas, 432 líneas no pasan por ninguna prueba. La cobertura de las condiciones es del 19.3%(134 condiciones sin cubrir) y la cobertura de líneas es del 17.9%. La gráfica de evolución temporal (Captura 7) muestra que mientras el código evaluable ha crecido a lo largo del sprint, las líneas cubiertas no han aumentado, provocando este déficit.
- **Deuda técnica y mantenibilidad [Capturas 1, 2, 3 y 6]:** En el apartado de incidencias hay 20 de tipo mantenibilidad, devolviendo una deuda técnica estimada en 2 horas y 4 minutos (Captura 1 y 2). La severidad de las incidencias son 8 medias, 10 bajas y 2 informativas. De las incidencias medias sobre todo las hay debido a códigos comentados no eliminados y malas prácticas de diseño como la falta de constructores privados en clases de módulo (Captura 2). La evolución temporal (Captura 6) muestra que el numero de incidencias aumenta conforme aumenta el volumen de proyecto (pasando de 12 a 20)
- **Complejidad ciclomática [Captura 4]:** El análisis de distribución muestra ficheros con una complejidad ciclomática alta (alcanzando valores de hasta 46) que presentan coberturas muy bajas o casi nulas (0-50%). Esto representa un punto crítico de riesgo para la mantenibilidad y evolución del código.

Plan de acción priorizado

El siguiente plan de acción establece la prioridad de resolución de los problemas encontrados, teniendo como preferentes las incidencias de mayor impacto en la mantenibilidad y al déficit de cobertura de pruebas:

PRI	INCIDENCIA	REGLA SONAR	ACCION CONCRETA	ESFUERZO ESTIMADO
1	Fichero con alta complejidad ciclomática y baja cobertura	Métricas de complejidad y cobertura	Refactorizar métodos extensos dividiendo responsabilidades y diseñar test unitarios sobre la clase	2h
2	Cobertura global de código insuficiente (18.2% actual, 432 líneas sin cubrir)	Cobertura de pruebas	Diseñar e implementar pruebas unitarias adicionales sobre las capas de lógica y repositorios	3h
3	Ausencia de constructor privado en clases de utilidad y módulos de inyección: RepositoriesModule.java (L25), Utils.java (L23) y CityBikesService.java (L10).	Java : S1118 Las Clases 'Utility' no pueden tener constructores públicos	Añadir un constructor privado explicito para ocultar el constructor público por defecto	15 min 3 x 5 min
4	Bloques de código comentado sin uso en MainPresenter.java (L180, L202), backup_rules.xml (L9) y archivo XML (L5).	Java : S125 Las secciones de código no deben estar comentadas	Eliminar las líneas y bloques de código comentados en desuso.	20 min 4 x 5 min

5	Variable local inútil / asignación redundante en ExampleTest.java (L22)	Java : S1481 / Java : S1854 Variables locales no utilizadas/asignadas	Eliminar la asignación y la variable local 'context' no utilizada.	5 min
6	Múltiples bloques de código con excesivo anidamiento en clases como StationsArrayAdapter.java (L61) y otros ficheros (L67, L69, L74, L75, L82)	Regla de buenas prácticas (Extract nested code block)	Refactorizar extrayendo bloques anidados a métodos auxiliares para mejorar legibilidad.	1h 6 x 10 min
7	Modificadores desordenados en CityBikesService.java (L13), import no usado en INetworksRepository.java (L3), redefinición de campo map en DetailsView.java (L32), uso de API legacy en lugar de java.time y comentario TODO pendiente en data_extraction_rules.xml (L8)	Reglas de estilo y convenciones Java	Reordenar modificadores, borrar import sobrante, declarar map como variable local, actualizar a java.time y revisar/resolver la tarea	30 min

- **PRI (Índice de prioridad)**: 1-2: Alta, 3-5: media, 6: baja. 7:baja/información.

Justificación de capacidad y tiempo disponible (Captura 5): Según el burndown chart de scumdesk, a fecha del informe restan aproximadamente 9.5 horas de trabajo disponible para completar el Sprint. La suma total del esfuerzo estimado para ejecutar todo el plan de acción es de 7 horas y 10 minutos. Por tanto, la aplicación de este plan es viable y asumible por el equipo dentro del margen temporal restante.

Tareas añadidas al Sprint

Para cumplir con el plan de acción dentro de la gestión de trabajo dentro de la gestión de trabajo del grupo, se han añadido las siguientes tareas al Backlog:

Tarea 1: Refactorización y pruebas de la clase de alta complejidad:

- Descripción: Refactorizar métodos con complejidad ciclomática elevada y añadir batería de pruebas unitarias
- Esfuerzo estimado: 2h

Tarea 2: Ampliación de cobertura de pruebas unitarias:

- Descripción: Implementar tests unitarios en Presenters y repositorios para elevar la cobertura general por encima del umbral deseado
- Esfuerzo estimado: 3h

Tarea 3: Corrección de incidencias de mantenibilidad media

- Descripción: Añadir constructores privados en 'RepositoriesModule', 'Utils' y 'CityBikesService', eliminar código comentado en 'MainPresenter' y ficheros XML de configuración y limpiar variables locales no utilizadas en 'ExampleTest'.
- Esfuerzo estimado: 40 min

Tarea 4: Refactorización de código anidado y code smells menores:

- Descripción: Extracción de bloques anidades en 'StationsArrayAdapter' y clases auxiliares, limpieza de import, ordenación de modificadores Java y migración de fechas a 'java.time'
- Esfuerzo estimado: 1h 30 min